SafeRoute IoT: An IoT-Based Adaptive Multi-Route Evacuation Guidance System for Educational Institutions

Sanket Chavan, Sanket Modak, Vaishnav Murade, and Sushant Adhe
Department of Computer Engineering, Pimpri Chinchwad College of Engineering (PCCOE), Pune, India

Abstract - Fixed emergency exit signs indicate directions chosen at design time and cannot respond when a corridor is blocked or an exit becomes unusable. This paper presents SafeRoute IoT, a proposed adaptive multi-route evacuation guidance system for educational buildings. The architecture links sensing nodes or simulated hazard inputs, publish-subscribe messaging, a backend holding the building graph, a route engine, light-emitting diode direction indicators, a student web interface, and an administrator dashboard. The building is modelled as a weighted graph whose edges and exits change state with hazard reports, and routes to all available exits are computed with a multi-source Dijkstra algorithm on the pruned graph. The paper distinguishes the ordinary shortest route, the minimum-cost feasible route, and the physically safe route, which the model cannot guarantee; it also specifies failure-handling rules and gives complexity and correctness arguments. An analytical worked example and a computational study on a synthetic three-floor graph show rerouting around blocked edges and closed exits. No physical hardware measurements, field trials, or evacuation outcomes are reported, and the results describe the routing algorithm only.

Index Terms - Adaptive routing, evacuation guidance, hazard-aware Dijkstra, Internet of Things

I. INTRODUCTION

Educational buildings such as colleges, schools, and laboratory blocks concentrate many occupants in multi-floor structures with corridors, staircases, and several exits. Occupants are often visitors or students who know only part of the layout, and laboratories can add further hazards. When an alarm sounds, the time available for a safe evacuation is limited, so occupants must quickly decide which way to move.

Conventional guidance relies on fixed exit signs that point toward predetermined exits and do not react to the state of the building. If the indicated corridor is affected by fire, smoke, or an obstruction, or if an exit is unusable, the sign can lead occupants toward a hazard. A recent simulation-based study of dynamic sign control used temperature and smoke data to set the direction of evacuation signs in a multi-floor, multi-exit shopping mall [1]. A survey of guided crowd evacuation discusses approaches and open challenges [2], and virtual-reality experiments have compared adaptive and fixed guidance [3].

The Internet of Things (IoT) makes it practical to collect hazard and route-condition data, transmit it with lightweight messaging such as MQTT [4], compute updated routes in software, and present them to occupants through physical indicators and personal devices. The two channels are complementary: indicators reach occupants who have no application installed, while a web page can show a full route and its status on a phone.

Research problem. Given a building graph whose edges and exits can become unavailable or less desirable over time, how can an IoT system select, for each location, a feasible route to an available exit, recompute it when conditions change meaningfully, and deliver the result to occupants, while behaving predictably when data are stale, communication fails, or no feasible route exists?

Contributions. This paper contributes (i) a graph-based formulation that separates the ordinary shortest route, the minimum-cost feasible route, and the physically safe route, and states exactly what safety means within the model; (ii) a layered architecture and data flow from hazard input to route delivery; (iii) an adaptive routing algorithm with failure-handling rules, pseudocode, and complexity analysis; and (iv) an analytical worked example and a computational study on a synthetic graph, kept strictly separate from any physical testing.

Scope and non-claims. We address the guidance function: turning hazard information and a building description into routes and moving them to occupants. We assume that a hazard indication arrives as a normalised level or a blocked flag, that the building graph is correct, and that occupants' start nodes are known or chosen. Sensor physics and fire alarm functions, which are governed by standards and approvals, are outside the scope. We do not claim that the system has been deployed or tested on physical hardware, that it shortens evacuation time, that occupants follow its guidance, or that it is safe in a regulatory sense.

II. Related Work

A. IoT-Enabled Evacuation Signage and Guidance

Prototype signage systems built on low-cost microcontrollers include a dynamic reactive signage network in which Dijkstra's algorithm runs on a server and a control signal sets the direction shown on an Arduino-driven RGB matrix panel [5], and an Arduino-based controller for smart LED exit signs [6]. A BIM-controlled signage system uses the signal returned by fire sensors to identify the fire zone and switches exit signs along a shortest route that avoids it [7]. Virtual-reality experiments indicate that adaptive emergency signs can guide people more efficiently than fixed ones [3]. The mechanism in [1] sets sign direction lights from temperature and smoke data using a congestion-aware heuristic evaluated through fire simulations. The descriptions of [3] and [5]–[7] used here follow the account given in [1].

B. Dynamic Routing and Shortest-Path Algorithms

Dijkstra's algorithm [8] finds minimum-cost paths in graphs with non-negative edge costs, and A* [9] accelerates single-target search with an admissible heuristic. Risk-aware evacuation work commonly assigns risk-based weights to links, using factors such as temperature, visibility, and carbon monoxide concentration, and then applies a shortest-path method [10], [11]. A comparison of safest and shortest paths reported that the safest route was often longer than the shortest one in the tested scenarios [12]. When several independent additive constraints must hold on one path, the routing problem is NP-complete in general [13]; SafeRoute IoT avoids this by using a per-edge hazard threshold and a single additive cost.

C. Multi-Floor Navigation and Hazard Sensing

In multi-floor buildings, staircases appear as edges linking nodes on adjacent floors, so evacuation becomes a routing problem over one connected graph, and stair congestion matters because many occupants may converge on few staircases [1], [14]. Systems using temperature and smoke sensors to detect fire and drive evacuation decisions are described in [1], [14], [15], and a prototype fire-detection and alert system based on temperature sensors is reported in [16]. Some signage studies use sensors only to indicate the presence or absence of an alarm rather than graded hazard levels [1]. MQTT is an open publish/subscribe transport that supports keep-alive and last-will mechanisms useful for detecting disconnected devices [4].

D. Comparison and Research Gap

Table 1 compares selected approaches with the proposed design. Entries for prior work are limited to what the cited material states, and the SafeRoute IoT entry describes the proposed design, not demonstrated performance.

Table 1. Selected approaches compared with the proposed design.

Work	Hazard input	Routing	Guidance output
[5]	Fire emergency signal	Dijkstra on server	RGB matrix panel (Arduino)
[6]	Fire detection (Arduino-based)	—	LED exit sign
[7]	Fire-sensor alarm signal	Shortest path avoiding fire zone	Switched exit signs
[3]	—	Routes avoiding fire zones	Adaptive signs (VR experiments)
[10], [11]	Temperature, visibility, CO	Shortest path on risk weights	—
[1]	Temperature, smoke	Heuristic with Dijkstra; congestion-aware	Sign direction lights (fire simulations)
This work (proposed)	Sensor or simulated hazard and blocked-edge status	Multi-source Dijkstra on pruned weighted graph	Web app; LED indicators where supported; dashboard

"—": not stated in the cited material. Prior-work entries follow the descriptions in [1].

Dynamic signage, risk-weighted shortest-path routing, and multi-floor graph models are established ideas. The narrower gap addressed here is a documented, end-to-end design for educational buildings that links an administrator-maintained floor-plan graph, MQTT-based hazard messaging, explicit failure-handling rules, and both student-device and indicator delivery. This does not imply that no comparable system exists; it reflects only the sources reviewed.

The closest work is the sign-control mechanism of [1], which adds a congestion index and evaluates a heuristic with fire simulations of a shopping mall. It also notes that route information sent only to phones reaches only occupants who have installed an application. Our design shares the aim of hazard-aware guidance and Dijkstra-type routing and keeps both a physical and a personal-device channel. Its cost model is deliberately simpler (a threshold, a weighted penalty, and an optional congestion term), and we do not claim that our routing is better than that of [1]; the evidence in [1] concerns a different building, hazard model, and algorithm, and does not transfer to this work. Related human-factors findings summarised in [1] suggest that a correct route can still fail if the instruction is not seen, understood, or trusted. This paper measures none of those factors and makes no claim about occupant compliance.

III. Proposed System and Methodology

A. Building Graph

The building is a connected undirected graph $G = (V, E)$. The node set V contains rooms, corridor junctions, stair landings, and exits; the edge set E contains corridors,

doors, stairs, and passages. Each edge $e = \{u, w\}$ has a type and a walking length $d_e > 0$ in metres (for stairs, an equivalent walking length). Let $X \subseteq V$ be the set of exit nodes. Time is divided into update epochs $t = 0, 1, 2, \ldots$, each started by a sensor message, an administrator action, or a timer.

B. Hazard, Congestion, and Availability

For every edge e the system maintains a normalised hazard level $\eta_e(t) \in [0, 1]$, derived from sensor readings or a simulated hazard; the mapping from readings to η is deployment-specific and not specified here. The blocked indicator $b_e(t) \in \{0, 1\}$ uses hysteresis with a lower threshold η_{lo} and an upper threshold η_{hi} (the blocking threshold, with $\eta_{\text{lo}} < \eta_{\text{hi}}$), and may be set manually by an administrator:

$$b_e(t) = 1 \text{ if } \eta_e(t) \geq \eta_{\text{hi}} \text{ or } e \text{ is manually blocked;} \quad (1)$$
$$b_e(t) = 0 \text{ if } \eta_e(t) \leq \eta_{\text{lo}} \text{ and not manually blocked;}$$
$$b_e(t) = b_e(t-1) \text{ otherwise, with } b_e(0) = 0.$$

An optional congestion level $\kappa_e(t) \in [0, 1]$ may be measured or simulated, and equals 0 if unavailable. The traversal cost of edge e is

$$c_e(t) = d_e(1 + \alpha\eta_e(t) + \beta\kappa_e(t)) \text{ if } b_e(t) = 0, \quad (2)$$
$$c_e(t) = \infty \text{ otherwise,}$$

where $\alpha, \beta \geq 0$ are penalty weights; with $\alpha = \beta = 0$ the cost equals the physical length. Exit availability is $a_x(t) \in \{0, 1\}$: an exit is unavailable if an administrator closed it, it is reported unusable, or all its incident edges are blocked. The available exit set is $X_a(t) = \{x \in X : a_x(t) = 1\}$.

C. Feasible Routes and Objective

A route from node v is a simple path $p = (e_1, \ldots, e_k)$ ending at some $x \in X_a(t)$. Its cost is $C(p, t) = \Sigma_{e \in p}\, c_e(t)$ and its length is $L(p) = \Sigma_{e \in p}\, d_e$. The feasible set $P(v, t)$ contains the routes that end at an available exit and use no blocked edge. The problem is to find

$$p^*(v, t) = \operatorname{argmin}_{p \in P(v,t)} C(p, t), \quad D(v) = C(p^*, t), \quad (3)$$

with $D(v) = \infty$ and a "no feasible route" status when $P(v, t) = \emptyset$. Feasibility is $\varphi(v) = 1$ if $P(v, t) \neq \emptyset$ and 0 otherwise. Equivalently, the cost-to-exit satisfies the recursion

$$D(x) = 0 \text{ for } x \in X_a; \quad (4)$$
$$D(u) = \min_{\{u,w\} \in E} (c_{uw} + D(w)) \text{ otherwise,}$$
$$N(u) = \operatorname{argmin}_{\{u,w\} \in E} (c_{uw} + D(w)), \quad (5)$$

where $N(u)$ is the next hop that an indicator or the web interface at node u should show. Because all destinations are available exits, one search from the set X_a yields D and N for every node.

D. Shortest, Minimum-Cost Feasible, and Physically Safe Routes

Three notions must be kept apart. The **ordinary shortest route** $p_d = \operatorname{argmin} L(p)$ on the intact graph ignores hazards and availability, which is what a fixed sign effectively encodes. The **minimum-cost feasible route** p^* of (3) differs in two ways: blocked edges and unavailable exits are excluded (hard constraints), and the weights α and β penalise hazardous or congested edges (soft constraints), so p^* may be longer than p_d even when no edge is blocked. The detour ratio

$$\rho = L(p^*) / L(p_d) \geq 1 \quad (6)$$

quantifies this trade-off, where p_d is the shortest route on the intact graph (with $\alpha = 0$ it coincides with the nominal minimum-cost route). The **physically safe route** is one that people could actually traverse without harm. The model cannot guarantee this, because it depends on the validity of η_{hi}, sensor accuracy, data freshness, and unmodelled factors such as crowd behaviour. We therefore use "feasible" for what the model computes: a route that is not flagged as blocked or unavailable under the configured thresholds. A cumulative-exposure constraint $\Sigma\, \eta_e d_e \leq H_{\max}$ would make the problem NP-hard in general [13] and is excluded from the baseline.

E. Properties of the Model

We state three properties for strictly positive finite edge costs, which hold whenever every $d_e > 0$. **Proposition 1 (optimality).** Dijkstra's algorithm [8], [17] on the pruned graph returns a minimum-cost route in $P(v, t)$ whenever it is non-empty and reports infeasibility otherwise. Optimality holds with respect to the cost model (2) only. **Proposition 2 (monotonicity).** Blocking an additional edge, closing an exit, or increasing any η_e or κ_e cannot decrease $D(v)$ for any node. *Proof:* each change removes routes from $P(v, t)$ or raises costs of those that remain, and a minimum over a smaller set or over costs that are no lower cannot be smaller. □ **Proposition 3 (loop-free guidance).** If $D(v) < \infty$, the sequence v, $N(v)$, $N(N(v))$, … reaches an available exit in at most $|V| - 1$ steps without repeating a node. *Proof:* by (4), $D(N(u)) = D(u) - c_{uN(u)} < D(u)$ for every non-exit node, so values strictly decrease, no node repeats, and the finite sequence can only stop at $D = 0$, an available exit. □ The guarantee requires the next-hop table to come from one consistent run; indicators applying entries from two table versions could loop, which is why Algorithm 1 attaches a version stamp to every published table. Proposition 2 also serves as a test oracle: any run in which $D(v)$ falls after a hazard is added reveals a bug.

F. Architecture and Data Flow

Figure 1 shows the proposed architecture. Sensors or simulated inputs injected through the administrator's hazard-simulation function provide hazard information. ESP32 nodes may read sensors, apply basic filtering, publish timestamped messages, and drive connected indicators. An MQTT broker carries sensor and route

messages between devices and the backend [4]. A Python backend (for example FastAPI) validates events, maintains hazard state, requests recomputation, and serves clients, while a database holds layouts, nodes, edges, exits, sensors, and route or hazard status. The route engine implements Section III-G and produces D, N, and φ. LED direction indicators at selected locations show the next hop, the student web application ("/navigate") presents guidance on personal devices, and the administrator dashboard supports floor-plan upload, exit and node management, sensor monitoring, and hazard simulation. The data flow is: a reading or event is published, forwarded to the backend, validated and mapped to an edge or exit state, routes are recomputed, the result is stored with a version stamp, and direction commands and web updates are delivered.

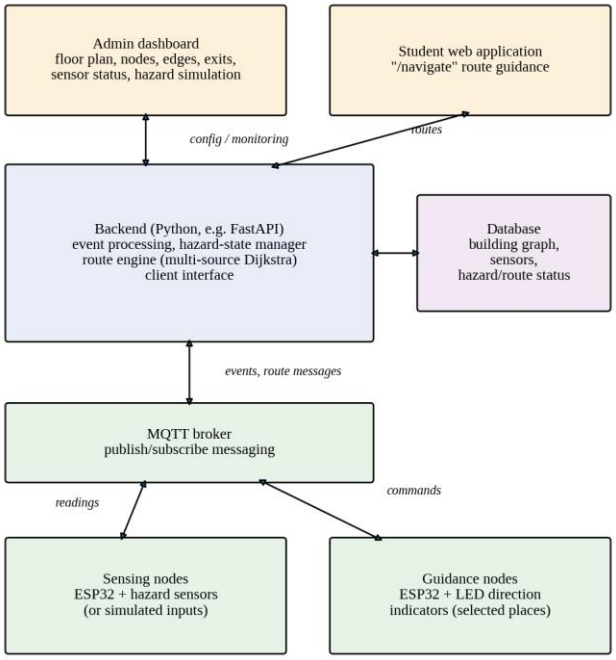


FIGURE 1. PROPOSED SYSTEM ARCHITECTURE. ARROWS SHOW THE PRINCIPAL DIRECTION OF DATA FLOW BETWEEN LAYERS. ALL BLOCKS DESCRIBE THE PROPOSED DESIGN; THEIR IMPLEMENTATION STATUS IS GIVEN IN TABLE 4.

A reading influences routing only after it is tied to a node or edge: a corridor sensor sets the hazard level of its edge, a junction sensor is propagated to incident edges by taking the maximum, and a door or exit sensor sets exit availability. Edges covered by no sensor must be declared by the administrator as assumed clear, assumed unknown (penalised), or manually controlled, and this choice changes what "feasible" means in any reported result. The end-to-end delay from event to updated instruction is decomposed as

$$T_{\text{total}} = T_{\text{sense}} + T_{\text{pub}} + T_{\text{proc}} + T_{\text{route}} + T_{\text{deliver}}, \qquad (7)$$

the sensing, transmission, event-processing, route-recomputation, and delivery delays. This paper does not measure any of these terms. Security matters because a false hazard message could close a good exit: authenticated and encrypted broker connections, per-device credentials, role-based dashboard access, and plausibility checks are minimum measures, and simulated events must be flagged and refused by live indicators outside an explicit test mode.

G. Adaptive Routing Algorithm

Because every destination is an available exit, the engine runs a single multi-source Dijkstra search from X_a on the pruned graph instead of one search per occupant (Algorithm 1). Dijkstra's algorithm suits non-negative costs and building-sized graphs; A* [9] needs coordinates for an admissible heuristic and serves one target at a time, Bellman–Ford [18] and Floyd–Warshall [19] handle negative costs and all-pairs queries that the model does not need. Ties are broken by node identifier so routes do not flip between recomputations.

Algorithm 1. Adaptive route update with failure handling.

Input: graph G, lengths d_e, exits X; parameters η_{lo}, η_{hi}, α, β, Δ, $\tau_{\min}$, T_{stale}; event stream
Output: next hop N, cost D, feasibility φ, with version stamp
1 initialise $\eta_e \leftarrow 0$, $\kappa_e \leftarrow 0$, $b_e \leftarrow 0$, $a_x \leftarrow 1$, $t_{\text{last}} \leftarrow -\infty$
2 on each event (sensor message, administrator action, timer):
3 validate; if invalid, log and return
4 update η, κ, overrides, exit flags, last-seen times
5 stale sensors (silent longer than T_{stale}): apply Table 2 policy
6 apply hysteresis, Eq. (1); update a_x for all x
7 if nothing changed, $\max|\Delta\eta| < \Delta$, no timer expiry: return
8 if now $- t_{\text{last}} < \tau_{\min}$: schedule deferred run; return
9 compute c_e by Eq. (2); $X_a \leftarrow \{x : a_x = 1\}$
10 if $X_a = \emptyset$: $\varphi(v) \leftarrow 0$ for all v; publish alert; return
11 $(D, N) \leftarrow$ MultiSourceDijkstra(G, c, X_a)
12 $\varphi(v) \leftarrow 1$ if $D(v) < \infty$ else 0; publish (N, D, φ, version); $t_{\text{last}} \leftarrow$ now
procedure MultiSourceDijkstra(G, c, X_a): $D(x) \leftarrow 0$ for $x \in X_a$, else ∞; queue $Q \leftarrow X_a$; while $Q \neq \emptyset$: $u \leftarrow$ ExtractMin(Q); for each neighbour w with $c_{uw} < \infty$: if $D(u) + c_{uw} < D(w)$: update $D(w)$, set $N(w) \leftarrow u$; return (D, N)

Parameters. The quantities below configure the operational behavior of SafeRoute IoT. Dimensionless penalty weights α and β scale the impact of hazardous and congested conditions; in this study, $\alpha \in \{0, 1, 3, 10\}$ is evaluated and congestion is taken as nominal ($\beta = 0$). For operational deployment, dual hazard thresholds are set with hysteresis: an upper blocking threshold ηhi = 0.65 and a lower unblocking threshold ηlo = 0.35 on the normalized scale, preventing high-frequency route oscillation under turbulent smoke conditions. The recomputation sensitivity threshold is set to $\Delta = 0.05$, and the minimum recomputation interval is configured to τmin = 1.0 s to throttle broker load. Sensor health monitoring marks any node silent longer than Tstale = 15.0 s as unknown, triggering the conservative penalty policy of Table 2. Indicator command acknowledgements timeout after Tcmd = 3.0 s, prompting administrative inspection while default physical exit signs remain operational.

Complexity. With a binary-heap priority queue one recomputation takes $O((|V| + |E|) \log |V|)$ time and $O(|V| + |E|)$ memory, independent of the number of occupants; reading the next hop is $O(1)$. Applying a single-source search once per node would cost a factor of $|V|$ more. These are asymptotic statements; no running times are reported. The multi-source search is equivalent to a single-source search from a virtual source joined to every available exit by a zero-cost edge, so Proposition 1 follows from the standard proof [8], [17]. The state logic around the search (hysteresis, suppression, deferral) is separate: a deferred recomputation must be scheduled so that later events cannot cancel it, otherwise the system could remain on an outdated table. The guarantee is conditional on the hazard data being true.

H. Failure Handling

Table 2 lists the failure cases and the intended policy for each: blocked corridors and unavailable exits are handled by the pruning in (2), and stale sensors, network failures, indicators that cannot update, and the absence of any feasible route have explicit rules. These are design choices that require validation, not claims of tested behaviour. If X_a is empty, no node has a feasible route and the system raises an alert instead of showing a direction.

TABLE 2. FAILURE CASES AND INTENDED HANDLING POLICIES (PROPOSED).

Failure case	Detection	Intended policy
Blocked corridor	$b_e = 1$ by Eq. (1)	Edge cost ∞; recompute routes
Exit unavailable	$a_x = 0$	Exclude from Xa; remove from the traversable graph (incident edges ∞), so it is not used as an intermediate node; recompute routes
Stale or missing sensor data	No message for longer than T_{stale}	Mark sensor "unknown"; do not treat the last "clear" reading as safe; apply the explicitly configured conservative policy (penalise or block the covered edge or exit, or require administrator confirmation); show the uncertainty on the dashboard
Device or network loss	MQTT keep-alive timeout or last-will message [4]	Treat as stale data; notify dashboard
Indicator cannot update	No acknowledgement within T_{cmd}	Fallback behaviour to be defined and tested; flag on dashboard; fixed signs remain
No feasible route	$\varphi(v) = 0$	Show "no feasible route" status; raise alert; follow institutional procedure
Out-of-order or duplicate messages	Version stamp or timestamp	Discard older messages; idempotent updates
Single-sensor error	Not detected by baseline	Administrator override; redundancy is future work

I. Illustrative Worked Example

Figure 2 and Table 3 give a small hand-checkable example on a two-floor graph with eight nodes. It is an analytical illustration computed from the model, not an experiment, and no sensor or hardware data are involved. The start node is room R2 on floor 2, and X1 and X2 are exits on floor 1. Scenario B shows that the ordinary shortest route is not always the minimum-cost route once hazard penalties apply, Scenario C shows hard blocking, and Scenario D shows infeasibility.

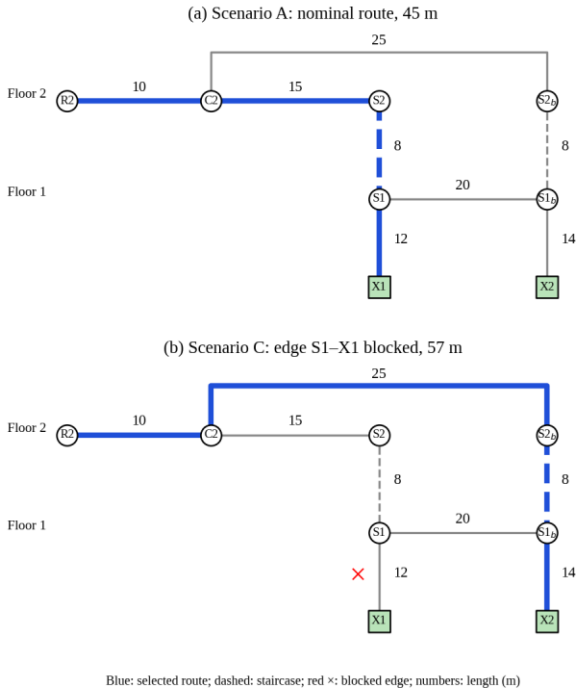


FIGURE 2. TWO-FLOOR EXAMPLE GRAPH. (A) NOMINAL ROUTE IN SCENARIO A. (B) REROUTING IN SCENARIO C AFTER EDGE S1–X1 IS BLOCKED. BLUE MARKS THE SELECTED ROUTE AND NUMBERS ARE EDGE LENGTHS IN METRES.

TABLE 3. WORKED EXAMPLE FROM R2, COMPUTED ANALYTICALLY FROM EQ. (2) WITH A = 3 (NOT MEASURED DATA).

Sc.	Edge or exit state	Selected route	Length (m)	Cost (m)
A	No hazard	R2→C2→S2→S1→X1	45	45.0
B	$\eta = 0.8$ on stairs S2–S1; nothing blocked	R2→C2→S2b→S1b→X2	57	57.0
C	Edge S1–X1 blocked	R2→C2→S2b→S1b→X2	57	57.0
D	S1–X1 blocked, exit X2 closed	none (no feasible route)	—	∞

Cost in metre-equivalents. In Scenario B the cost of S2–S1 is 8 × (1 + 3 × 0.8) = 27.2, so the route through X1 costs 64.2 although it is the ordinary shortest route (45 m).

IV. Implementation and Experimental Setup

A. Status of Components

This paper distinguishes four levels of evidence: proposed features, implemented software, simulations and computational experiments, and physical hardware testing. The source material lists the components below as the proposed architecture and does not state which are built or tested, so Table 4 records only what is supported. The only executed software reported here is a script that applies the routing algorithm and cost model to a synthetic graph. No physical hardware testing, sensor-accuracy measurement, or latency measurement is reported.

TABLE 4. EVIDENCE STATUS OF EACH COMPONENT.

Component	Level	Status in this paper
Route engine (multi-source Dijkstra, Eq. (2))	Computational experiment	Executed in a standalone script on a synthetic graph (Section V); prototype integration not reported
ESP32 sensing nodes; sensors	Proposed	Sensor models, firmware, and sampling not specified; simulated inputs are an alternative
MQTT messaging	Proposed	Broker, topics, and quality-of-service settings not specified
Backend; database	Implemented software	State management, dynamic graph pruning, and event pipeline implemented in backend service.
Student web app; admin dashboard	Implemented software	Responsive student guidance portal ("/navigate") with local navigation and administrator monitoring dashboard fully implemented in interactive web prototype.
LED direction indicators	Proposed	Selected locations where supported; no physical testing reported
Hazard simulation	Proposed (dashboard); script used here	Study injects blocked edges and exit closures directly in software

B. Computational Experiment Setup

The synthetic graph has three floors. Each floor has a main corridor of eight junctions and a parallel secondary corridor of eight junctions, joined by three cross-links, with one room attached to every main-corridor junction. Three staircases link adjacent floors and three exits lie on floor 1. The graph has 75 nodes and 84 edges. The 24 rooms are start nodes, and 57 corridor, cross-link, and stair edges are candidates for blocking; door edges are not blocked, since blocking one merely isolates a room or exit. Corridors are 8 m, cross-links 6 m, stairs 10 m, room doors 4 m, and exit doors 5 m. Construction: junctions J*f*_i and K*f*_i for floors $f = 1, 2, 3$ and positions $i = 0, \ldots, 7$; rooms attached to J*f*_i; cross-links at $i = 0, 3, 7$; staircases at J*f*_0, J*f*_7, and K*f*_3; exits X1, X2, X3 joined to J1_0, J1_7, and K1_3. Ties are resolved by node name.

Four experiments were run. (1) Each of the 57 candidate edges was blocked in turn with $\alpha = 0$, giving 1368 edge–room cases. (2) Every combination of closed exits was evaluated. (3) For k = 1, …, 12, k edges were drawn uniformly without replacement from the 57 candidates, with 2000 draws per k and fixed seed 12345 (Python's random.Random applied to candidate edges deterministically sorted by edge identifier with $\alpha = 0$). (4) With nothing blocked, $\eta = 0.5$ was set on one edge at a time and routes were compared with the nominal ones for $\alpha \in \{0, 1, 3, 10\}$. All numbers are exact counts from deterministic or seeded computation. Running times are not reported, because a time measured on the authoring machine says nothing about the target platform. Physical hardware testing, if later undertaken, must use simulated messages or approved test devices, must not disable or contradict statutory exit signs, and needs safety-officer and, for participants, ethical approval.

V. Results and Discussion

All results in this section are computational experiments on the synthetic graph. They describe the routing algorithm, not the prototype, the hardware, or the network, and they are not field trials or physical evacuations.

A. Single Blocked Edge

Of the 1368 edge–room cases, the nominal route used the blocked edge in 51. All 51 found an alternative route, so no single blocked edge left a room without a route. Over the rerouted cases the mean detour ratio ρ was 1.49 and the largest was 3.00. In the remaining 1317 cases the nominal route did not use the blocked edge, and in all 1317 the minimum cost was unchanged, as Proposition 2 and the locality of cost increases require.

B. Exit Closure

Table 5 lists every combination of closed exits. Because all three exits are reachable from every room in the intact graph, closing one or two exits leaves every room with a route and changes only its length; closing X1 and X3 raises the mean ratio to 1.96. Closing all three exits is the only case in which the system must report that no route exists.

TABLE 5. EXIT CLOSURE ON THE SYNTHETIC GRAPH (COMPUTED; 24 START ROOMS; $\alpha = 0$).

Closed exit(s)	Rooms with a route (%)	Mean ρ (rooms with a route)
X1	100	1.32
X2	100	1.45
X3	100	1.15
X1, X2	100	1.77
X1, X3	100	1.96
X2, X3	100	1.94

Closed exit(s)	Rooms with a route (%)	Mean ρ (rooms with a route)
X1, X2, X3	0	—

ρ is the cost with the exits closed divided by the nominal cost, which equals the length ratio for α = 0.

C. Random Multi-Edge Blocking

To see how the picture changes when several edges fail together, k edges were blocked at random. Table 6 and Figure 3 give the mean share of rooms that still have a route, the share of draws in which every room still has one, and the mean ratio ρ over reachable rooms. The mean share of reachable rooms falls slowly, from 100.0% at $k = 1$ to 86.9% at $k = 12$, whereas the share of draws that leave every room reachable falls quickly, to 17.1% at $k = 12$.

TABLE 6. RANDOM MULTI-EDGE BLOCKING (2000 DRAWS PER K, SEED 12345; COMPUTED; SELECTED K, ALL VALUES IN FIGURE 3).

k blocked edges	Rooms with a route (%, mean)	Draws with all rooms reachable (%)	Mean ρ
1	100.0	100.0	1.02
2	99.9	98.5	1.04
4	99.2	88.9	1.08
6	97.9	75.7	1.12
8	95.2	54.8	1.17
10	91.8	33.1	1.20
12	86.9	17.1	1.23

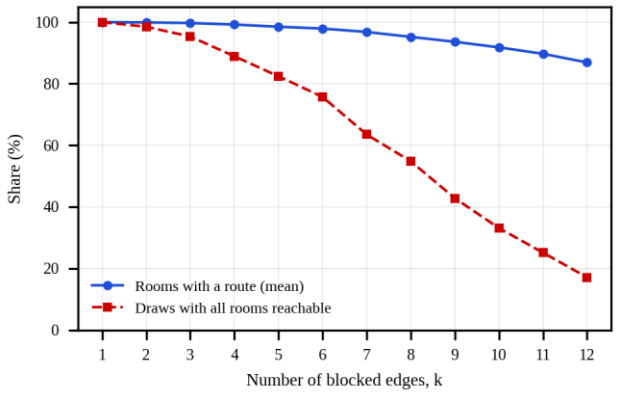


FIGURE 3. RANDOM MULTI-EDGE BLOCKING ON THE SYNTHETIC GRAPH (DATA OF TABLE 6 AND THE OMITTED K VALUES). THE BLUE CURVE SHOWS THE MEAN SHARE OF ROOMS WITH A ROUTE; THE DASHED RED CURVE SHOWS THE SHARE OF DRAWS IN WHICH EVERY ROOM REMAINS REACHABLE.

D. Sensitivity to the Hazard Penalty

Finally, nothing was blocked and $\eta = 0.5$ was set on one candidate edge at a time. Table 7 shows that the penalty has little influence at $\alpha = 1$ and grows with α, but even at $\alpha = 10$ the average share of rooms whose route changes stays below 4%, because a single hazardous edge matters only to rooms whose route uses it and who have a reasonable alternative.

TABLE 7. EFFECT OF A SOFT HAZARD PENALTY (H = 0.5 ON ONE EDGE, NOTHING BLOCKED; COMPUTED).

α	Rooms with changed route (%, mean over edges)	Mean route-length ratio	Edges with any change (%)
0	0.00	1.000	0.0
1	0.66	1.000	14.0
3	1.97	1.004	36.8
10	3.73	1.018	42.1

The ratio compares the physical length of the new route with the nominal one, averaged over the 24 rooms and the candidate edges.

E. Discussion

Table 3 and Figure 2 show why the three route notions differ: the ordinary shortest route (45 m through X1) becomes the costlier choice under a hazard penalty on its stairs, and becomes unusable when its last edge is blocked. Tables 5 and 6 quantify the price of feasibility on one graph: a single blocked corridor or staircase was always bypassed, at a mean of about half as long again and, in the worst case, three times as long. Figure 3 shows that a single building-wide "evacuation possible" flag would hide rooms that have lost their routes when many edges fail, which is a reason to publish feasibility per node, as Algorithm 1 does. Table 7 shows that soft penalties alter routes only for a minority of rooms in this graph.

These findings depend strongly on the graph: the synthetic building has three exits, a second corridor on every floor, and three staircases, and a building with one long corridor and two staircases would behave very differently. The numbers therefore illustrate the method and are not evidence about any real building. System-level questions, namely how quickly a hazard report becomes a visible instruction and what happens when messages are lost, remain open until physical or bench measurements are made.

F. Limitations

Sensor accuracy and availability bound the quality of η and b, and false negatives or positives can leave a hazardous edge open or block a useful one. Occupant positions are assumed known or selected. Network, broker, or power failures delay updates, and indicator fallback behaviour is unvalidated. Crowd behaviour and congestion are not modelled dynamically, stair edges use equivalent lengths and ignore capacity, accessibility needs are not enforced, and thresholds and weights are the authors' choices and change routes. Route feasibility has not been validated against fire-engineering criteria, and the system cannot substitute for institutional emergency procedures or regulations. Dynamic indicators would supplement, not replace, statutory signs, which must remain usable if the dynamic part fails.

VI. Conclusion and Future Work

SafeRoute IoT is proposed as an adaptive evacuation guidance system for educational institutions that replaces fixed directions with routes computed from a building graph whose edges and exits change with hazard status. The paper formalised the feasible-route problem, separated the ordinary shortest, minimum-cost feasible, and physically safe routes, specified an algorithm with $O((|V| + |E|) \log |V|)$ cost per recomputation, and defined failure-handling rules. The evidence reported is an analytical example and a computational study of the routing algorithm on a synthetic graph; no physical hardware measurements, field trials, or evacuation outcomes are reported, so the safety value of the system remains to be demonstrated. Future work includes implementing and documenting the prototype interfaces, bench measurement of the latency terms in (7) with synchronised clocks, richer cost models with calibrated exposure and visibility terms, sensor redundancy, congestion-aware routing with capacity-limited stairs and exits, accessibility-aware paths, battery-backed fail-safe indicators, and, with fire-engineering review and ethical approval, controlled trials in real buildings.

Declarations

Author contributions: S. Chavan, S. Modak, V. Murade, and S. Adhe conceptualized the study, developed the graph routing model, implemented the web prototype and hardware simulation pipeline, designed and executed the computational experiments, and drafted the manuscript. All authors reviewed and approved the final paper.

Funding: This research received no external grant from any funding agency in the public, commercial, or not-for-profit sectors.

Ethics approval and consent: Not applicable. The computational experiments reported herein involved no human participants, animal testing, or personal data; ethical approval and informed-consent procedures were therefore not required.

Data and code availability: The synthetic building graph specification, automated evaluation scripts, and prototype software codebase are openly available in the project repository.

Acknowledgments: The authors thank the faculty and laboratory coordinators of the Department of Computer Engineering, Pimpri Chinchwad College of Engineering (PCCOE), for academic guidance and laboratory computing facilities.

Conflicts of interest: The authors declare that they have no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] H.-H. Yen and C.-H. Lin, "Intelligent evacuation sign control mechanism in IoT-enabled multi-floor multi-exit buildings," *Sensors*, vol. 24, Art. no. 1115, 2024, doi: 10.3390/s24041115.

[2] M. Zhou, H. Dong, P. A. Ioannou, Y. Zhao, and F.-Y. Wang, "Guided crowd evacuation: Approaches and challenges," IEEE/CAA J. Autom. Sinica, vol. 6, no. 5, pp. 1081–1094, Sep. 2019, doi: 10.1109/JAS.2019.1911672.

[3] H. Zhao et al., "Fire evacuation supported by centralized and decentralized visual guidance systems," Saf. Sci., vol. 145, Art. no. 105451, Jan. 2022, doi: 10.1016/j.ssci.2021.105451.

[4] A. Banks and R. Gupta, Eds., *MQTT Version 3.1.1*, OASIS Standard, Oct. 29, 2014. [Online]. Available: docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[5] C. S. Baidal, N. X. Arreaga, and V. S. Padilla, "Design and testing of a dynamic reactive signage network towards fire emergency evacuations," Int. J. Electr. Comput. Eng., vol. 10, no. 6, pp. 5853–5860, Dec. 2020, doi: 10.11591/ijece.v10i6.pp5853-5860.

[6] J. Jung, D. Kim, and H. Lee, "IoT enabled smart emergency LED exit sign controller design using Arduino," Int. J. Adv. Smart Converg., vol. 6, no. 4, pp. 76–81, Dec. 2017, doi: 10.7236/IJASC.2017.6.4.76.

[7] K. Yenumula, C. Kolmer, J. Pan, and X. Su, "BIM-controlled signage system for building evacuation," Procedia Eng., vol. 118, pp. 284–289, 2015, doi: 10.1016/j.proeng.2015.08.428.

[8] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[9] P. E. Hart et al., "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[10] J. Zhang, J. Guo, H. Xiong, X. Liu, and D. Zhang, "A framework for an intelligent and personalized fire evacuation management system," Sensors, vol. 19, no. 14, Art. no. 3128, Jul. 2019, doi: 10.3390/s19143128.

[11] Y. Hu and X. Liu, "Optimization of grouping evacuation strategy in high-rise building fires based on graph theory and computational experiments," IEEE/CAA J. Autom. Sinica, vol. 5, no. 6, pp. 1104–1112, Nov. 2018, doi: 10.1109/JAS.2018.7511228.

[12] H. Bayat, M. R. Delavar, W. Barghi, S. A. EslamiNezhad, P. Hanachi, and S. Zlatanova, "Modeling of emergency evacuation in building fire," Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci., vol. XLIII-B4-2020, pp. 313–320, Aug. 2020, doi: 10.5194/isprs-archives-XLIII-B4-2020-313-2020.

[13] Z. Wang and J. Crowcroft, "Quality-of-service routing for supporting multimedia applications," *IEEE J. Sel. Areas Commun.*, vol. 14, pp. 1228–1234, 1996.

[14] H.-H. Yen, C.-H. Lin, and H.-W. Tsao, "Novel smoke-aware individual evacuation and congestion-aware group evacuation algorithms in IoT-enabled multi-story multi-exit buildings," IEEE Access, vol. 10, pp. 119402–119418, 2022, doi: 10.1109/ACCESS.2022.3221330.

[15] H.-H. Yen et al., "Time-aware and temperature-aware fire evacuation path algorithm in IoT-enabled multi-story multi-exit buildings," *Sensors*, vol. 21, Art. no. 111, 2021, doi: 10.3390/s21010111.

[16] F. Idris, N. Hashim, A. F. Kadmin, and L. B. Yee, "Intelligent fire detection and alert system using LabVIEW," Int. J. Electr. Comput. Eng., vol. 9, no. 3, pp. 1842–1849, Jun. 2019, doi: 10.11591/ijece.v9i3.pp1842-1849.

[17] T. H. Cormen et al., *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[18] R. Bellman, “On a routing problem,” *Q. Appl. Math.*, vol. 16, pp. 87–90, 1958.

[19] R. W. Floyd, “Algorithm 97: Shortest path,” *Commun. ACM*, vol. 5, p. 345, 1962.